Project Report

Face Mask Detection using Computer Vision

---

1. Introduction

With the outbreak of global health crises such as COVID-19, wearing face masks became essential to prevent virus transmission. However, ensuring compliance in public spaces like offices, colleges, and transportation hubs is challenging.

This project presents a **real-time face mask detection system** using **Computer Vision and Deep Learning** to automatically identify whether a person is wearing a mask.

---

2. Problem Statement

Manual monitoring of mask usage:

- Is **time-consuming**
- Requires **human supervision**
- Is prone to **errors and negligence**

Problem:

How can we automatically detect whether people are wearing face masks in real-time using computer vision?

---

3. Why This Problem Matters

- Helps improve **public safety**
- Reduces dependency on manual monitoring
- Useful in **crowded environments**
- Can be integrated into **smart surveillance systems**

---

4. Proposed Solution

We developed a system that:

1. Captures real-time video from a webcam
2. Detects faces using a pre-trained model
3. Classifies each face as:
    - With Mask
    - Without Mask
4. Displays results with bounding boxes and labels

---

5. Methodology

Step 1: Data Collection

- Used dataset with two classes:
    - `with_mask`
    - `without_mask`

Step 2: Data Preprocessing

- Resized images to **224x224**
- Normalized pixel values
- Converted images into arrays

Step 3: Model Selection

- Used **MobileNetV2 (Transfer Learning)**
- Lightweight and efficient for real-time applications

Step 4: Model Training

- Frozen base layers of MobileNetV2
- Added custom classification layers
- Trained using labeled dataset

Step 5: Face Detection

- Used **OpenCV DNN-based face detector**
- Detected faces in real-time video frames

Step 6: Prediction

- Extracted face region
- Preprocessed input
- Passed to trained model
- Displayed prediction

---

6. Tools & Technologies

- Python
- OpenCV
- TensorFlow / Keras
- NumPy
- MobileNetV2

---

7. System Architecture

Workflow:

1. Webcam Input
2. Face Detection (OpenCV DNN)
3. Face Extraction
4. Preprocessing
5. Mask Classification (Deep Learning Model)
6. Output Display

---

8. Key Design Decisions

- Used **MobileNetV2** for faster inference
- Used **Transfer Learning** to reduce training time
- Chose **224x224 input size** for compatibility
- Used **batch prediction** for performance optimization

---

9. Challenges Faced

1. Dataset Issues

- Incorrect folder structure caused errors
- Fixed by organizing dataset properly

2. Path Errors (Windows)

- Faced `unicodeescape` errors
- Solved using `/` instead of `\`

3. Model Loading Error

- Keras 3 did not support `.model` format

- Fixed by using `.h5`

4. Git & Deployment Issues

- Git not installed / PATH issues
- Solved by proper installation

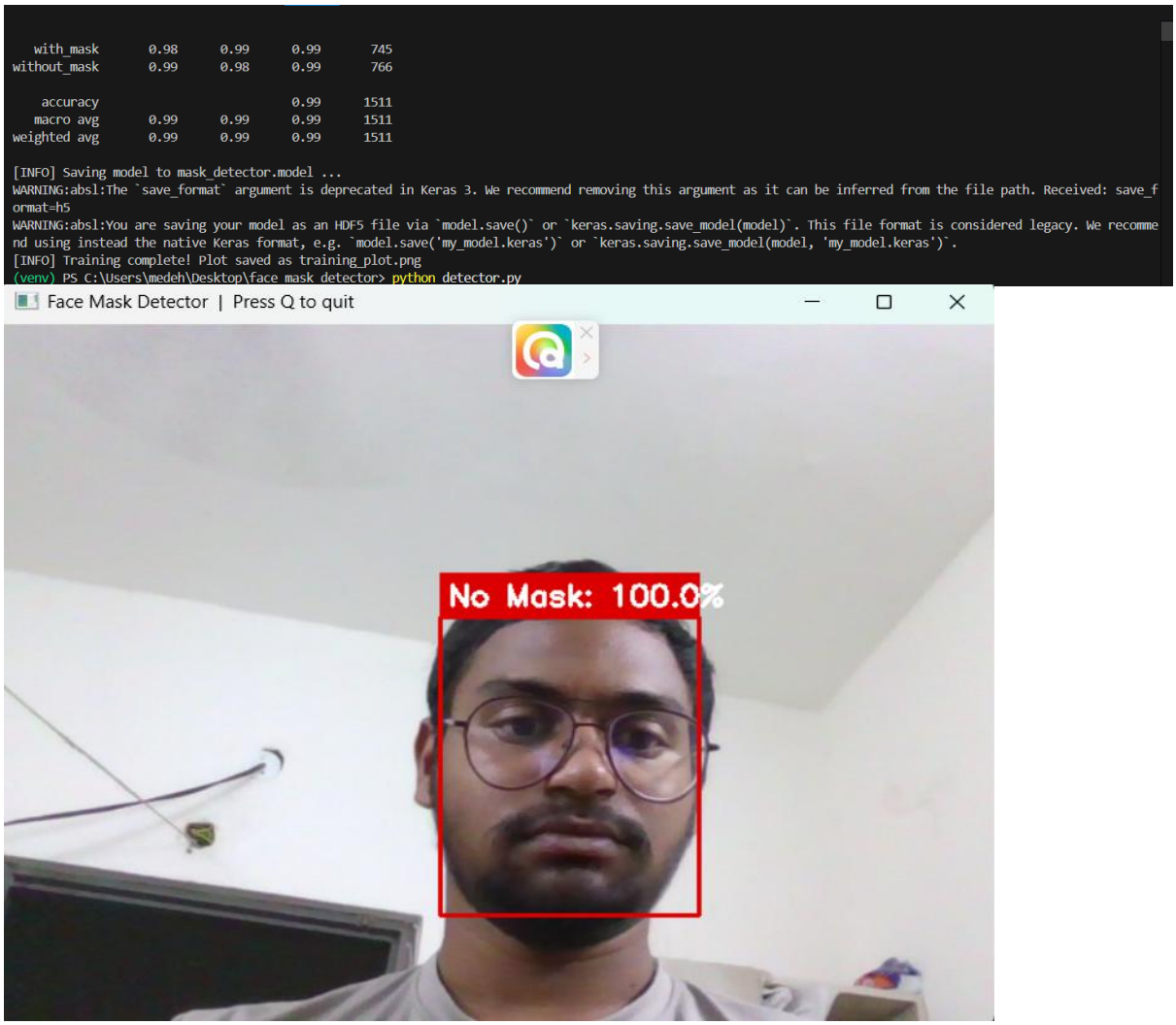
5. Real-time Performance

- Needed optimization for smooth detection

---

10. Results

- Successfully detected faces in real-time
- Correctly classified mask vs no-mask
- Achieved good accuracy with small dataset



11. Output Description

- Green box → Mask detected
- Red box → No mask detected
- Confidence score displayed

---

12. Limitations

- Performance depends on lighting conditions
- May struggle with:
  - Partial face visibility
  - Low-resolution images
- Limited dataset reduces generalization

---

13. Future Improvements

- Add **face recognition (identity tracking)**
- Deploy as **web/mobile application**
- Improve accuracy with larger dataset
- Add **alert system (sound/email)**
- Integrate with CCTV systems

---

14. Learning Outcomes

Through this project, I learned:

- Practical implementation of **Computer Vision**
- How to use **Transfer Learning**
- Real-time video processing using OpenCV
- Debugging real-world issues (paths, models, environment)
- GitHub version control and project deployment

---

15. Conclusion

This project demonstrates how computer vision can be used to solve real-world problems like enforcing mask usage. The system is efficient, scalable, and can be extended to many safety and surveillance applications.

Author

Mittapally Sri Harsha
GitHub: https://github.com/HarshaMittapally